



プログラム説明資料 ガマズミ

タイトル「ガマズミ」

チーム制作

4年生作品

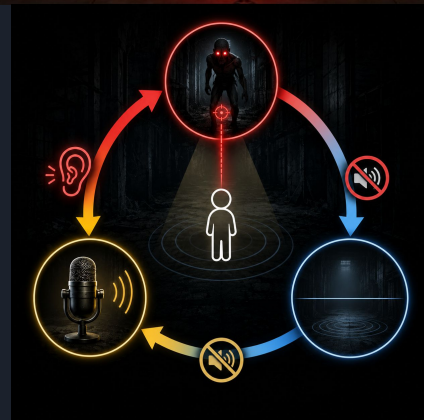
ジャンル：3Dステルスホラー脱出ゲーム

開発環境: Unity/ Visual Studio/C#

制作期間：約4ヶ月

【作品概要】

「現実世界の音が命を分かち」没入型ホラー。マイク入力による音声検知と、敵の接近時に環境音が消失する独自の演出を組み合わせた、**五感を刺激する**ステルス体験を提供。





操作説明

W / A / S / D	: 移動
マウス	: 視点移動
左Shift	: ダッシュ
E	: 懐中電灯のON / OFF
右クリック	: スマホUIのON / OFF
左クリック	: インタラクションイベント
Esc	: ポーズ画面の表示

担当箇所

本プロジェクトでは **リーダー兼プログラマー** として
開発全体の進行管理を担当しました。開発2か月目に、
コア機能である音声解析システムの実装が難航し、
全体スケジュールが約2週間遅延する課題が発生しました。

そこで私は、機能の優先順位を見直し、
ステージの複雑化など優先度の低い要素を削減する判断 を行いました。
その分の開発リソースを **音声解析システムのデバッグと改善に集中**させることで、
プロジェクト全体の立て直しを図りました。

技術面では、プレイヤー操作、敵 AI、音声解析、アイテムシステム、UI制御、
シーン遷移、謎解きギミックなど主要システムの設計・実装を担当しました。
また、探索導線や謎解き配置の調整にも携わり、
ゲーム体験の向上に取り組みました。

その結果、ホラーゲームの核となる「声を出すと見つかる」体験の品質を維持したまま、
作品を予定通りに導くことができました。この経験を通して、状況に応じて
優先順位を見極め、プロジェクトを動かす判断力を身に付けました。



核心技術

自律型索敵 AI

敵がプレイヤーを見失った瞬間に巡回へ戻ってしまい、追われている**恐怖や緊張感が持続**しないという問題がありました。

有限状態機械を用いた敵AIを設計し、
Search状態とLast Known Positionの記憶機能を実装しました。
プレイヤーを見失った後も最後に確認した場所を
重点的に探索することで、
人間らしい粘り強い索敵行動を実現しています。また、
DOTweenによる視線移動や追跡猶予時間を導入し、
発見されそうな緊張感を演出しました。

隠れても安全ではなく、「すぐ近くで敵が探している」というステルスゲーム特有の緊張感を実現しました。さらに、NavMeshによる自然な経路探索とAnimator Hash IDによる負荷軽減により、**没入感と実行効率を両立**しています。



リアルタイム音声解析システム

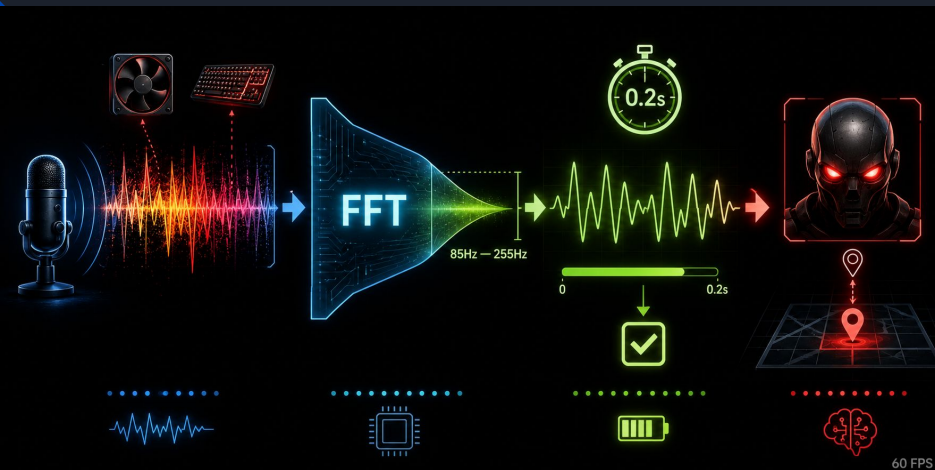
プレイヤーの声で敵に見つかるホラー体験を実現したかった、一方で**単純な音量判定**では**PCのファン音**や**キーボード音**にも**反応**し、「**排熱音でゲームオーバーになる**」という理不尽な問題がありました。

UnityのOnAudioFilterReadで音声データを取得し、**FFT**(高速フーリエ変換)による周波数解析 を実装しました。

マイク入力→FFT変換→音声帯域(85Hz~255Hz)抽出
→ノイズ除去・平滑化→AIへ通知

さらに、一定時間以上継続した
信号のみを有効とすることで誤検知を抑制 しました。

検知した音声を敵 AIの状態機械と連携させ、
「声を出すと見つかる」緊張感 を実現しました。
60FPSを維持したまま 高精度な音声認識を行い、
ゲーム性と技術的完成度を両立した
音声解析システムを構築しました。



```
private void ProcessVoiceInput()
{
    // 平均化
    volumeRate = Mathf.Lerp(
        volumeRate,
        targetVolume,
        smoothSpeed * Time.deltaTime);

    // ノイズゲート
    if (volumeRate < noiseThreshold)
    {
        volumeRate = 0f;
    }

    // 音声帯域判定
    float voiceBandVolume = GetVoiceBandVolume();

    bool voiceDetectedNow =
        volumeRate > noiseThreshold &&
        voiceBandVolume > 0.001f;

    // 0.2秒継続判定
    if (voiceDetectedNow)
    {
        voiceTimer += Time.deltaTime;
        if (voiceTimer >= voiceDetectTime)
        {
            isVoiceDetected = true;
        }
    }
    else
    {
        voiceTimer = 0f;
        isVoiceDetected = false;
    }
}
```

```
private void ShowSoundSource()
{
    if (!isEnemyCanHear) return;

    // マイク感度の閾値
    if (!isVoiceDetected) return;

    if (volumeRate < 1f) return;

    Vector3 playerPos = player.position;

    float distance =
        Vector3.Distance(
            enemy.position,
            player.position);

    hearingDistance =
        25f - (distance / 60f * 10f);

    hearingDistance =
        Mathf.Clamp(
            hearingDistance,
            3f,
            25f);

    SetHeardPoint(new Vector3(playerPos.x, 0f, playerPos.z));

    hideTimer = 0f;

    seAudio.mute = false;
    isSoundSourceActive = true;

    if (!soundSource.activeSelf)
    {
        soundSource.SetActive(true);
    }
}
```

継承を用いた相互作用システム

本作品では、ギミック追加によるコードの複雑化を防ぎ、拡張性の高いシステムを構築することを目指しました。従来はドアや鍵、ロッカーなどのギミックが増えるたびに**プレイヤー側の判定処理へ条件分岐**を追加する必要があり、**保守性の低下やバグ発生**の原因となっていました。そこで、

すべてのインタラクティブオブジェクトの基底クラスとして「InteractionEvents」を作成し、共通インターフェースによる相互作用システムを実装しました。**プレイヤーは対象の種類を意識せず**「目の前のオブジェクトへ干渉する」という共通処理のみを実行し、**各ギミック固有の挙動は継承先クラス**で管理しています。

その結果、ドア・鍵・ロッカーなどの**新機能を既存コードを変更することなく追加可能**となり、**保守性と拡張性を両立した設計**を実現しました。

```
using UnityEngine;
using UnityEngine.UI;

// Unity スクリプト 10 個の参照
public class Key : InteractionEvents

{
    public Text countText;
    public GameObject objectToShow;
    public PINItemDatabaseScript PINItemDatabase;
    public InventoryUI inventoryUI;
    public int itemIndex = 0;
    //name
    string itemName;
    // Start is called before the first frame update
    // Unity メッセージ 10 個の参照
    void Start()
    {
        itemName = PINItemDatabase.PINList[itemIndex].itemName;
    }
    // 2 個の参照
    public override void interactionEvents()
    {
        this.gameObject.SetActive(false);
        countText.text = "残り " + (PINItemDatabase.PINList[itemIndex].count - 1);
        objectToShow.SetActive(true);
        inventoryUI.UpdateItemUI(itemIndex);
    }
}
```

```
using UnityEngine;
using UnityEngine.UI;

// Unity スクリプト 10 個の参照
public class DoorScript : InteractionEvents

{
    public bool isOpen = false; // 扉が開いているかどうか
    public bool isTouching = false; // 扉が動いているかどうか
    // Start is called before the first frame update
    // Unity メッセージ 10 個の参照
    void Start()
    {
        // Update is called once per frame
        // Unity メッセージ 10 個の参照
        void Update()
        {
            // 1 個の参照
            public override void interactionEvents()
            {
                OpenAndClose();
            }
        }
    }
    // 開け始め
    // 4 個の参照
    public void OpenAndClose()
    {
        if (isTouching == false) return;
        if (isOpen == false) OpenDoor();
        else CloseDoor();
    }
}
```

ScriptableObjectによる 柔軟なコンテンツ管理と保守性向上

本作品では、**アイテム情報や暗証番号、ヒントテキストの調整を効率化し、**
チーム全体の開発速度を向上させることを目指しました。
開発当初は**データをスクリプトへ直接記述**していたため、
内容を変更するたびに
「プログラム修正 → コンパイル → 再ビルド」が必要となり、
細かなバランス調整に時間がかかることが課題でした。

そこで、アイテム名や説明文、画像、暗証番号などを
ScriptableObjectとして外部化し、Unityエディタ上から
管理できる仕組みを構築しました。また、
インベントリUIにはアイテムIDのみを保持させ、
必要な情報をデータベースから取得することで、
UIとデータを分離した疎結合な設計を実現しています。

その結果、従来は数分かかっていた調整作業をInspector上で
数秒で行えるようになり、プログラマ以外のメンバーでも
ゲームバランスやテキストを編集可能になりました。
これにより、**開発効率・保守性・拡張性を向上**させ、
チーム全体の生産性向上に貢献しました。

```
public class ItemUI : MonoBehaviour
```

```
//アイテムのUIを管理するクラス
```

```
[SerializeField]  
public Image itemImage; //アイテムの画像を表示するUIコンポー  
[SerializeField]  
public Button itemButton; //アイテムを選択するUIコンポーネン
```

```
[SerializeField]  
public int item; //アイテムのデータを保持する変数  
// Start is called before the first frame update
```

```
//アイテムのUIを表示するメソッド
```

```
1 個の参照  
public void ShowItemUI(Sprite sprite)
```

```
itemImage.sprite = sprite; //アイテムの画像を表示するUI  
itemImage.GetComponent<Image>().enabled = true; //アイテ  
itemButton.interactable = true; //アイテムを選択するUIコ
```

```
public class InventoryUI : MonoBehaviour
```

```
//インベントリのUIを管理するクラス
```

```
[SerializeField]  
public List<ItemUI> inventory;  
[SerializeField]  
public PINiteDatabase PINIiteDatabase;  
//アイテムのUIを管理するクラス  
[SerializeField]  
public Image itemImage; //アイテムの画像を表示するUIコンポーネン  
[SerializeField]  
public Text itemNameText; //アイテムの名前を表示するUIコンポーネン  
[SerializeField]  
public Text itemDescriptionText; //アイテムの説明を表示するUIコンポーネン
```

```
//アイテムのUIを更新するメソッド
```

```
1 個の参照
```

```
public void UpdateItemUI(int item)
```

```
//アイテムのデータを取得する
```

```
if (item < 0) return; //アイテムが存在しない場合は処理を終了する  
if (item > inventory.Length) return; //アイテムのインデックスが範囲外の場合は処理を終了する  
InventoryItem itemData = PINIiteDatabase.PINIet(item).searchName;  
inventory[item].item = item; //アイテムのインデックスをアイテムのUIに代入する
```

```
//アイテムのUIを調べるメソッド
```

```
0 個の参照
```

```
public void CheckItemUI(List<ItemUI> list)
```

```
//メソッド
```

```
0 個の参照
```

```
public void CheckItemUI(List<ItemUI> list)
```

```
if (list == null) return;  
if (list.Count == 0) return; //アイテムのインデックスを取得する  
if (list.Count == 0) return; //アイテムが存在しない場合は処理を終了する  
InventoryItem itemData = PINIiteDatabase.PINIet(list[0].searchName); //アイテムの画像を表示するUIコンポー  
itemImage.sprite = PINIiteDatabase.PINIet(list[0].searchName).searchImage; //アイテムの名前を表示するUIコンポー  
itemNameText.text = PINIiteDatabase.PINIet(list[0].searchName).searchName; //アイテムの説明を表示するUIコンポー
```



物理・時間・入力を統合した 快適なゲーム体験の実現

本作品では、**プレイヤーが恐怖体験に集中** できるよう、**操作性・安定性・パフォーマンス**を支える**基盤システム**を設計しました。

シーン遷移には **LoadSceneAsync** を用いた**非同期ロード**を採用し、ロード中の画面停止 を防止しています。
また、入力制御や状態管理を統一することで、**連続入力や想定外の操作** による不具合を抑制しました。

プレイヤー移動には **Rigidbody** を利用し、**加速度を考慮した自然な挙動**を実現しています。さらに、**スタミナシステムには疲労状態** を導入し、**ホラーゲーム特有の緊張感** を演出しました。

加えて、**DOTweenのコールバック**を活用して演出と**ゲームロジックを同期**し、**状態遷移時の不整合** を防止しています。
これらの設計により、**快適な操作性と高い安定性**を実現し、**ゲーム体験を支える堅牢なシステム** を構築しました。

```
// ロードを開始するメソッド
// 7 個の参照
public void StartLoad(string sceneName)
{
    StartCoroutine(Load(sceneName));
}

// コルーチンを使用してロードを実行するメソッド
// 1 個の参照
private IEnumerator Load(string sceneName)
{
    // ロード画面を表示する
    loadingUI.SetActive(true);

    // シーンを非同期でロードする
    asyncOperation = SceneManager.LoadSceneAsync(sceneName);

    // ロードが完了するまで待機する
    while (!asyncOperation.IsDone)
    {
        loadingObject.transform.Rotate(0, 0, -360 * Time.deltaTime);
        yield return null;
    }

    // ロード画面を非表示にする
    loadingUI.SetActive(false);
}
```

```
//スタミナの消費と回復を処理する
// 6 個の参照
private void HandleStamina()
{
    bool usingSprint =
        Input.GetKey(KeyCode.LeftShift) &&
        Input.GetKey(KeyCode.LeftSpace) > 0.5f &&
        movement.velocity.magnitude > 0.1f &&
        canSprint;

    if (usingSprint)
    {
        ConsumeStamina();
    }
    else
    {
        RecoverStamina();
    }
}

//スタミナの回復
// 6 個の参照
void ConsumeStamina()
{
    if (!canSprint) return;
    if (!isNearThreshold) return;
    useStamina += Time.deltaTime;
    recoverStamina = 0f;

    if (useStamina > staminaRecoverInterval)
    {
        stamina = Mathf.Max(0, stamina - 1);
        useStamina = 0f;

        if (stamina <= 0)
        {
            canSprint = false;
            UpdateUI();
        }
    }
}

//スタミナの回復
void RecoverStamina()
{
    recoverStamina += Time.deltaTime;
    useStamina = 0f;
    if (recoverStamina > staminaRecoverInterval)
    {
        stamina = Mathf.Min(maxStamina, stamina + 1);
        recoverStamina = 0f;

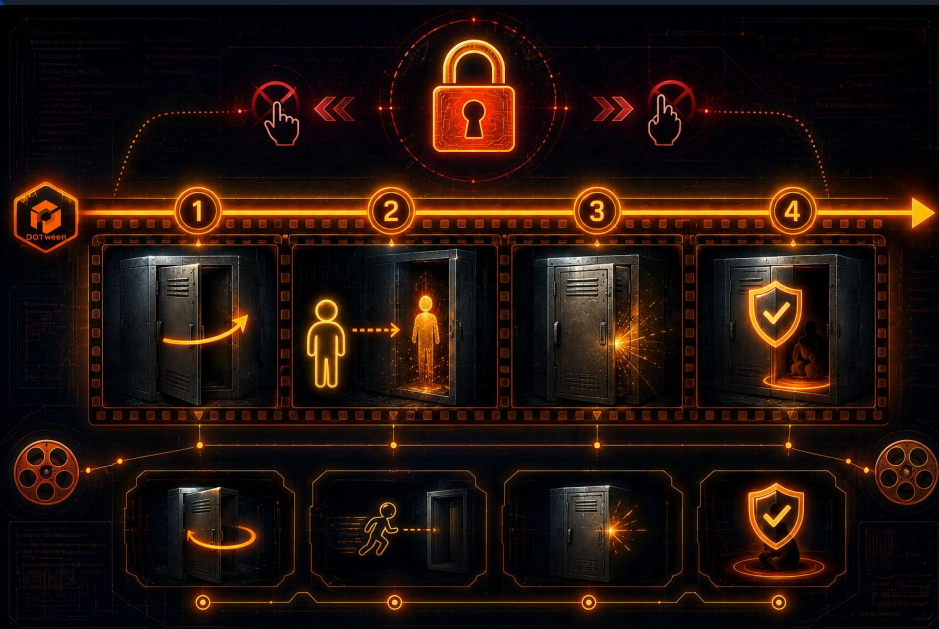
        if (stamina >= fatigueRecoverBorder)
        {
            canSprint = true;
            UpdateUI();
        }
    }
}
```

ロッカー隠脱システムのシーケンス制御

本作品では、プレイヤーがロッカーへ隠れる際の没入感と安定した操作性の両立を目指しました。しかし開発当初は、扉を開く演出中にプレイヤーが移動できてしまい、壁へのめり込みや状態不整合が発生するという課題がありました。

そこで、DOTweenのSequenceを用いて「扉を開く→ロッカーへ移動→扉を閉じる→隠れ状態へ移行」の一連の処理を単一のタイムライン上で制御しました。さらに、isTouchingフラグによる排他制御を導入し、連続入力による二重実行や状態破綻を防止しています。

その結果、演出と物理判定を完全に同期させることで、不自然な挙動のない安定した隠脱体験を実現しました。プレイヤーの没入感を損なわず、ホラーゲーム特有の緊張感を維持できるシステムとなっています。



```
public class Jlocker : DoorScript
{
    BoxCollider boxCollider;
    public MeshCollider hitJudgment;
    GameObject player;
    Transform behindTheDoor;
    public Transform inFrontOfTheDoor;
    //Start is called before the first frame update
    @Unity メッセージ10 個の参照
    void Start()
    {
        boxCollider = GetComponent<BoxCollider>();
        player = GameObject.FindGameObjectWithTag("Player");
        behindTheDoor = transform.parent.gameObject.transform;
    }
    2 個の参照
    public override void CloseDoor()
    {
        EventsDesignedToReleaseThePlayer();
    }
    1 個の参照
    public override void OpenDoor()
    {
        EventsDesignedToHideThePlayer();
    }
}
```

```
//プレイヤーを扉の奥に隠すためのイベント
void EventDesignedToHideThePlayer()
{
    var sequence = DOTween.Sequence();
    boxCollider.enabled = false;
    isTouching = true;
    player.GetComponent<Player>().isHiding = true;
    //扉をあけてプレイヤーを扉の奥に隠してから扉を閉める
    sequence.Append(transform.DORotate(new Vector3(0, -90, 0), 1f, RotateMode.LocalAxisAdd));
    Append(player.transform.DOLocal Move (new Vector3(behindTheDoor.position.x, player.transform.position.y, 0), 1f));
    AppendCallBack(() =>
    {
        //プレイヤーを扉の方に向ける
        player.transform.rotation = Quaternion.Euler(0, behindTheDoor.rotation.eulerAngles.y, 0);
    });
    AppendInterval(1);
    Append(transform.DORotate(new Vector3(0, 90, 0), 1f, RotateMode.LocalAxisAdd));
    AppendCallBack(() =>
    {
        isOpen = true;
        isTouching = false;
        boxCollider.enabled = true;
    });
}
```

```
//プレイヤーを扉から出すためのイベント
void EventDesignedToReleaseThePlayer()
{
    var sequence = DOTween.Sequence();
    boxCollider.enabled = false;
    isTouching = true;
    sequence.Append(transform.DORotate(new Vector3(0, -90, 0), 1f, RotateMode.LocalAxisAdd));
    Append(player.transform.DOLocal Move (new Vector3(inFrontOfTheDoor.position.x, player.transform.position.y, 0), 1f));
    AppendCallBack(() =>
    {
        //プレイヤーを扉の方に向ける
        player.transform.rotation = Quaternion.Euler(0, inFrontOfTheDoor.rotation.eulerAngles.y, 0);
    });
    AppendInterval(1);
    Append(transform.DORotate(new Vector3(0, 90, 0), 1f, RotateMode.LocalAxisAdd));
    AppendCallBack(() =>
    {
        isOpen = false;
        isTouching = false;
        boxCollider.enabled = true;
        player.GetComponent<Player>().isHiding = false;
    });
}
```



開発を通じて得た知見



① 抽象化による拡張性の確保

ギミックごとに個別実装を行うのではなく、共通インターフェースによる疎結合設計を採用しました。その結果、中盤以降の仕様変更や新機能追加にも既存コードへの影響を最小限に抑えて対応できました。今後も**保守性と拡張性を意識した設計**を実践していきます。

② パフォーマンスと体験品質の両立

敵AIの更新頻度や探索処理を分析し、計算負荷とゲーム体験のバランスを検討しました。必要な箇所へ計算資源を集中させることで、プレイヤーの体感品質を維持しながら最適化を実現しました。この経験から、**技術だけでなくユーザー体験を基準に判断する重要性**を学びました。



③ チーム全体の生産性向上

ScriptableObjectを活用したデータ駆動設計を導入し、
非プログラマでもゲームバランスや
コンテンツを調整できる環境 を構築しました。
自分がコードを書くことだけでなく、
**チーム全体が成果を出しやすい仕組みを作ることも
プログラマの重要な役割** だと実感しました。

総括

本プロジェクトでは、技術課題の解決だけでなく、
設計・最適化・チーム開発の観点から課題 へ向き合いました。
これらの経験で培った課題解決力を活かし、
貴社でも**早期に戦力として貢献していきたい** と考えています。

タスク管理表					
内容	担当	状況	着手日	期限日	期限
般若		対応中	4/21	6/30	2 日
自室(現代)		完了	4/21	5/1	
ベット		完了	2/21	5/1	
机		完了	2026/04/24	5/1	
イス		完了	2026/04/24	5/1	
鍵		完了	2026/04/22	5/1	
手帳		完了	2026/04/22	5/1	
懐中電灯モデル		完了	2026/05/06	5/1	
タンス		完了	2026/05/08	2026/05/15	
神社		対応中	2026/06/15	2026/07/10	12 日
神具 三宝		対応中	2026/06/15	2026/07/03	5 日



今後の改善案

1. 設計・保守性の向上

- マジックナンバーを排除し、ScriptableObjectやSerializeFieldで管理
- InteractionEventsをインターフェース化し、より疎結合で拡張しやすい設計へ改善

2. 音声解析の高度化

- 機械学習を活用し、人の声と生活音をより高精度に判別
- マイクごとの設定を保存し、環境差による誤検知を低減

3. 敵AIの発展

- FSMからBehavior Treeへ移行し、より複雑で賢い行動を実現
- 壁や部屋の構造を考慮した音の伝播システムを導入

今後の展望

今後は音声解析への機械学習導入やAIの高度化を進め、より没入感の高いゲーム体験を追求したいと考えています。
チーム全体の生産性向上に貢献できるエンジニアを目指します。

自分のポートフォリオの

URL: https://docs.google.com/presentation/d/1tHu_XddIDycfcXOa1eMRN7laMpt0fVPC4AxSMrS3xA/edit?slide=id.p#slide=id.p